

NALMOBench: 面向移动对象数据库自然语言接口的基准测试

王谢阳 南京航空航天大学
，中国 xieyang@nuaa.edu.cn

魏嘉 南京航空航天大学
，中国 wji_x@nuaa.edu.cn

刘萌怡* 中国南京航空航天大学
大学 liumengyi@nuaa.edu.cn

陈晨 宗 南京航空航天大学
，中国 chencz@nuaa.edu.cn

摘要

现有的数据库自然语言接口（NLIDs）基准主要针对关系数据库，缺乏统一的质量指标、对复杂时空多维查询的支持，以及对翻译准确性和效率的真实世界验证。为填补这一空白，我们提出了一个面向移动对象数据库（MODs）的自然语言查询（NLQ）翻译基准。该基准涵盖五类常见的移动对象查询以及两个真实场景查询。这些NLQ通过使用大型语言模型（LLMs）生成，以构建语料库。从语料库中提取出稳健的NLQ，并在自动化流程和领域专家的协助下进行标注。标注过程中融入形式化验证，以确保查询质量。我们提出了一种两阶段查询转换基线方法，包括自然语言理解和结构化语言生成，用于理解NLQ的意图并生成优化的可执行查询。

VLDB 研讨会参考格式：

王谢杨、易伟佳、刘梦怡、宗晨晨。NALMOBench: 迈向移动对象数据库自然语言接口的基准测试。VLDB 2025 研讨会：LLM+Spatial。

1 介绍

近年来，时空数据库的发展是由对位置感知数据的复杂管理日益增长的需求所推动的。自然语言，作为最

熟悉的交流方式，已成为非专业人士使用数据库的首选方式。实际上，基于大语言模型（LLM）的文本到 SQL 技术的进步促进了面向特定领域数据库的自然语言接口的发展。然而，标准 SQL 无法捕捉用于深层时空语义推理的丰富时空操作符，而原生时空引擎则提供了广泛的可执行语言集合，可以表达复杂的查询 [12]。为弥合这一差距，研究人员开始探索从自然语言直接翻译为可执行语言（text-to-EXE）。然而，这一新兴领域缺乏统一的基准和公共数据集。参考文本到 SQL 基准的成熟度，我们需要讨论以下三个问题。

数据库：关于柏林模拟列车运动轨迹的数据库，包括51,544个移动对象点。

Example NLQ: Find the 5 continuous nearest neighbors to the train 100 between 6:00 and 19:00 o'clock.

GPT-4o predicted SQL:

```
SELECT A.Id
FROM Trains AS A, Trains AS B
WHERE B.Id = 100
AND StartPoint(Intersection(A.Trip, B.Trip)) BETWEEN mpoint('2020-11-20
06:00:00') AND mpoint('2020-11-20 19:00:00')
ORDER BY Distance(StartPoint(A.Trip), StartPoint(B.Trip))
LIMIT 5;
```

Executable language of SECONDO:

```
let t100 = Trains feed filter[.Id = 100] extract[Trip];
query UTOOrdered_rtree UTOOrdered feed filter [(deftime(UTrip) intersects [const
periods value (('2020-11-20-06:00" "2020-11-20-19:00" TRUE TRUE))]]]
nearest_neighbor[UTrip, t100, 5] consume;
```

图 1: GPT-4o 预测的 SQL 示例及相应的 SECONDO 可执行语言。白色部分是实体。加粗且带下划线的部分是由于解释最近邻结构语义表达的运算符。

Q1: 相比于文本到SQL，对于移动对象数据库，我们为什么需要文本到EXE？尽管取得了显著进展，但大多数成果集中在关系数据库上，例如基于规则的方法NaLIR [6]、基于神经网络的方法IRNet [3]，以及基于大语言模型的方法如FinSQL [16]。使用SQL进行查询的关系数据库在遇到

*Corresponding author

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment. ISSN 2150-8097.

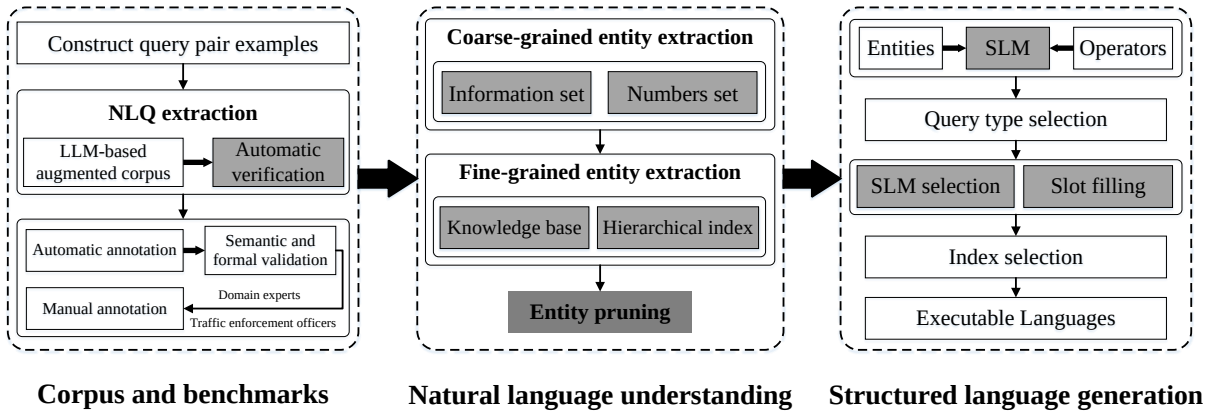


图 2: 基准概览。

处理涉及多维数据的复杂移动对象查询。现有的自然语言接口数据库 (NLIDBs) 通常依赖优化器进行查询优化, 这可能导致对数据和性能的控制能力下降。相比之下, 可执行语言由于支持广泛的时空查询和操作符, 提供了解决语义解析和查询转换的可能性, 并且可以通过索引使用和操作符选择直接优化查询。如图1所示, GPT-4生成的SQL需要综合使用多种操作符和谓词才能准确表示最近邻查询的语义。可执行语言仅使用nearest_neighbor操作符即可表达相同的语义。此外, 可执行数据库查询可以选择性地利用由UTOrdered_rtree表示的3D R树索引。请注意, 后续的可执行语言示例均采用SECONDO [4] 的可执行语言, 该系统是一个真实的多维对象数据库。

Q2: LLM能否主导文本到EXE的领域? 如前所述, 基于LLM的方法现在主导了文本到SQL的领域, 这使我们考虑LLM是否可以有效用于文本到EXE。然而, LLM在生成可执行语言时面临两个主要障碍。一方面, 现有的方法依赖于可执行语言的特定任务语料库, 这通常需要提示工程或使用新创建的数据集进行昂贵的微调。依赖提示会在实体链接、操作符选择和索引使用的准确性之间引入权衡。此外, 为微调设计和创建专门的语料库通常成本高昂。另一方面, 即使使用提示或微调, 文本到SQL领域的实体链接准确率仍低于75%, 更不用说复杂的时空查询了。为了全面研究现有的翻译方法和LLM是否能够为文本到EXE提供有价值的见解

第三问: 如何评估文本到 EXE 系统? 主流基准主要关注文本到 SQL 的评估, 未能解决 MOD 的复杂时空查询需求。早期数据集如 Geoquery [15] 仅提供自然语言查询 (NLQs), 没有对应的 SQL。虽然 WikiSQL [18] 提供了 NLQ 和 SQL 配对, 但仅适用于简单查询。Spider [14] 和 Bird [7] 支持跨领域。

查询但目标固定数据集, 缺乏对复杂MOD查询的专业评估。MOD查询通常代表现实世界的需求。目前的基准存在两个基本局限性: (i) 文本到SQL的关注忽略了特定领域文本到可执行查询的评估, 以及(ii) 现有评估缺乏现实移动对象场景验证, 过度强调翻译准确性而低估了可执行查询的效率。这些差距需要专门的基准来捕捉特定领域的特征和语义, 并配合一个平衡准确性和效率的综合评估框架。

挑战。为了在MOD的背景下构建一个深入的文本到EXE可能的基准, 我们应当应对三个主要挑战: (i) 缺乏基于领域知识的基准构建。MOD的复杂领域知识以及广泛的可扩展操作符给自然语言查询 (NLQs) 的构建和标注带来了困难。(ii) 自然语言理解的难度。空间和时间信息都可能具有歧义。在设计查询时, 需要考虑同义词、缩写和模糊实体。(iii) 结构化语言生成的复杂性。由于操作符的多样性和查询类型的丰富性, 结构化语言生成必须同时解决实体与操作符的组合以及查询优化的挑战。

为了解决这些局限性, 我们提出了NALMOBench, 这是一个基于大型语言模型 (LLM) 的基准, 用于评估MODs中的文本到EXE的转换能力。该复杂的真实世界基准评估文本到EXE系统和LLM的性能。我们提出的基于LLM的自动语料生成方法能够高效地生成大规模带注释的文本到EXE数据集, 并具备领域泛化能力, 同时解决自然语言理解和结构化语言生成的挑战。我们还提供了用于比较的基线方法。据我们所知, NALMOBench是首个由时空数据库专家和交通执法人员共同开发的基准。该基准全面支持标准动态对象查询, 包括时间区间查询、范围查询、最近邻查询、连接查询和轨迹相似性查询, 以及真实世界的交通执法场景。在现有基准中独树一帜的是,

翻译准确性和执行效率。图2展示了基准架构。贡献总结如下：

- 我们提出了 NALMOBench, 一个新的基准平台, 旨在评估文本到 EXE 系统和用于 MOD 的大型语言模型 (LLM), 该平台包含超过 600 个自然语言与可执行数据库查询的配对, 涵盖标准操作和真实世界场景, 分为 7 种查询类型, 并按 3 个难度级别分层。
- 我们通过一种混合方法构建了大约 6,000 个移动对象查询的语料库, 该方法结合了人工收集的自然语言查询 (NLQs) 和基于大语言模型 (LLM) 的自动数据增强, 并结合了语义和形式验证。
- 我们通过分别构建 NLQ 候选和数据库候选来进一步优化自然语言理解算法。我们提出了一种分而治之的范式来优化结构化语言生成。
- 我们在 NAL-MOBench 上评估了不同的 NLIDB 系统和 LLM。实验结果表明, 基线方法在可翻译性、翻译准确性和翻译效率得分方面分别达到 68.96%、60.07% 和 68.37, 优于现有方法。

2 相关工作

基准在NLIDB系统的开发和验证中起着关键作用。高质量的基准可以促进NLIDB研究的积极发展和持续进步。早期关于NLIDB基准的研究通常使用专有数据集, 这些数据集针对特定领域量身定制。例如, 基于解析的系统如NaLIR、PRECISE [10] 和 ATHENA++ [11] 依赖于如Geoquery、MAS 和 YELP等数据集, 这些数据集专注于单个领域, 尽管在各自领域表现良好, 但缺乏通用性。

随着神经网络技术的发展, 关于自然语言到数据库接口 (NLIDB) 的研究对系统的泛化能力提出了更高的要求。尽管基于神经网络的技术不断涌现, 但缺乏可在跨领域情境中使用的测试数据集。因此, 提出了WiKiSQL, 这是第一个面向关系数据库的大规模多领域数据集。WiKiSQL包含80,654条自然语言问题 (NLQs) 和77,840条SQL语句。然而, WiKiSQL中的SQL语句相对简单, 缺乏排序、分组和子查询等复杂操作。因此, Spider通过构建一个用于跨领域语义分析的大型复杂NLIDB数据集, 进一步解决了WiKiSQL的局限性。Spider包含10,181条NLQs、5,693条SQL语句, 并涉及138个不同领域中的200多个数据库。数据集中既有常见的简单SQL查询, 也有嵌套查询等复杂查询。尽管WiKiSQL和Spider都提出了跨领域的数据集, 这两个基准仍然强调数据库模式。

随着大型语言模型 (LLM) 时代的到来, 自然语言接口数据库 (NLIDB) 的研究迎来了新的范式。Bird被提出用来评估 LLM。Bird包含12,751对文本到SQL的匹配对和95个数据库, 总规模为33.4 GB, 涵盖37个专业领域。Bird提出

一种用于评估自然语言到数据库系统 (NLIDB) 效率的验证指标, 尽管主要关注时间成本。

虽然 Spider 和 Bird 宣称具有很高的泛化能力, 但其中大多数数据库是由学生专门创建的, 这可能无法充分代表现实世界的数据库复杂性。ScienceBenchmark [17] 是第一个设计用于复杂现实世界科学数据库的文本到 SQL 基准。现实世界的应用很可能需要在单一领域中实现高性能, 这可能比跨领域的泛化能力更为重要。

3 基准建设

现有的基准测试主要面向文本到 SQL 的任务, 没有专门为文本到 EXE 情境构建的基准测试。虽然声称具有跨领域适用性, 但这些基准测试未能充分满足特定领域的需求, 例如移动对象查询。此外, 大多数基准测试缺乏在真实场景中的验证。为了解决这些限制, 我们开发了 NALMOBench, 专门用于 MOD 中的文本到 EXE 验证。算法 1 显示了基准测试构建的工作流程。

Algorithm 1: Corpus Construction and Annotation

```

Input : the list of  $n$  NLQs,  $Q$ ;
          the schema information of the database,  $S$ ;
          moving objects NLQ corpus,  $MOC$ ;
          the list of  $n$  executable database queries,  $EL$ ;
Output : validated query pairs,  $P$ 
          initialize  $Q$  with collected NLQs
for  $q_i \in Q$  do
   $el_i \leftarrow$  manual annotation of  $q_i$ 
  Models construction according to  $Type(q_i)$ 
for  $Model \in Models$  do
   $q \leftarrow Model(S)$ 
   $Q.append(q)$ 
for  $q_i \in Q$  do
  if  $q_i$  fails to meet the semantic, syntax, and entity
  constraints then
     $Q.remove(q_i)$ 
for  $q_i \in Q, el_i \in EL$  do
   $el_i \leftarrow NALMO(q_i)$ 
  if  $z3(el_i) \wedge semantic(el_i) = semantic(q_i)$  then
     $EL.append(el_i)$ 
 $P.append(Q, EL)$ 
return  $P$ 

```

3.1 语料库构建方法

我们开发了一种基于模板的方法, 利用收集的自然语言查询 (NLQs), 并通过大语言模型 (LLMs) 增强, 以构建一个包含 6,000 条关于移动对象的自然语言查询的预处理语料库, 涵盖时间区间查询、范围查询、最近邻查询、连接查询、轨迹相似性查询、跨区域查询和绕行查询。

3.1.1 自然语言查询收集。我们收集并设计了100个稳健的运动物体自然语言查询 (NLQs)。对于每种类型的查询, 除真实世界查询外, 我们从期刊中收集并手动完善了80个自然语言查询。

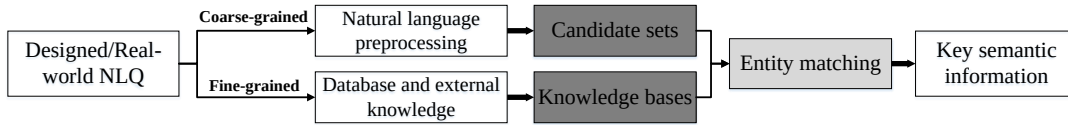


图 3：自然语言理解。

和会议。我们假设来自顶级数据库期刊和会议的查询是高质量的。现实场景中的查询由交通执法人员提出，随后由专家修改，这反映了实际场景，并由专家进一步微调以符合研究背景。此外，我们手动标注了这100个自然语言查询，详细的标注过程将在后文描述。

3.1.2 自动增强。在收集的查询基础上，我们为每种查询类型开发了先进的自然语言查询（NLQ）增强模板。由于篇幅限制，我们展示了范围查询的一个模板：“哪些 <对象> 会在 <位置类型> <特定位置> 之间从 <开始时间> 到 <结束时间> 经过？”这些模板作为使用单次示例（one-shot）方法构建相应提示（prompt）的基础。然后，我们利用 GPT-4o 扩展语料库，从而丰富 NLQ 的表达多样性。相应的提示如图4所示。

```

### 仅完成 SECONDO 可执行语言查询且不附带解释 ### SECONDO 可执行语言表格及其属性：##模式：
{模式信息} #根据给定模板，填写被掩盖的槽位以生成自然语言查询，形成{特定查询类型}查询。模板：
在时间段 [<start_time>, <end_time>] 内，找到 <object1> <object_id1/object_identification1> 与 <object2> <
object_id2/object_identification2> 之间的相同轨迹。生成的查询：

```

图4：使用自然语言模板进行自然语言生成的提示。

3.1.3 问题审查。为了进一步确保自然语言问题（NLQ）的质量，我们进行二次验证流程，使用链式思维（CoT）方法设计提示，让 GPT-4o 验证每个查询的以下方面：（i）语法是否正确。（ii）实体信息是否与查询类型对应。（iii）是否存在任何歧义。对于不合规的查询，由专家进行审查。由于在基准构建过程中还会进行进一步的 NLQ 提取，初步验证的主要重点是确保查询内容的语法正确性和稳健性，而不与数据库信息进行交互。

3.2 基准构建方法

我们采用结合手动与自动方法的混合方式来构建自然语言查询（NLQ）与可执行语言对。最初，我们手动创建了100对关于移动物体查询的稳健对。随后，我们使用自动验证方法从现有语料库中提取约500条NLQ。此外，我们利用 NALSpatial [8] 中给出的方法进行初始注释，然后使用语法和...对注释查询进行验证。

形式化验证技术。最后，我们依赖专家来标注无法自动注释的查询。

3.2.1 手动生成。我们聘请了三位专门研究时空数据库的研究生和两名交通执法人员参与查询对的手动构建。非现实场景的验证由这三位研究生进行。跨区域和绕行操作相关的查询由执法人员提出。研究生随后根据数据库对这些查询进行微调，并构建相应的查询对，这些查询对通过执行可执行数据库查询由执法人员进行验证。总共构建了100个查询对。同时，我们利用构建的语料库提取自然语言查询（NLQs），并使用 GPT-4o 通过带有组织示例的链式推理（CoT）自动化验证过程来提取 NLQs。

3.2.2 注释。我们使用 NALSpatial 提供的方法进行初始注释。然而，该方法只能生成部分类型的查询。因此，我们使用手工构建的示例对，并利用 GPT-4o 模型评估生成的可执行语言与原始自然语言查询（NLQs）之间的语义等价性。随后，我们进行 Z3 形式验证，这包括解析可执行语言以提取查询类型、结构和组成部分。为实体创建符号变量，并将不同子句类别中的条件转换为 Z3 约束，以检查约束的一致性，特别是用于识别和解决查询约束中的任何矛盾。对于无法生成或未通过验证的查询，我们会请领域专家调整或重建可执行数据库查询。特别地，所有现实场景的查询均为手工构建。

4 自然语言理解

自然语言的广泛语义和多样的句法变体阻碍了机器的直接理解，导致实体识别和提取方面存在重大困难。为了解决这些挑战，我们开发了一个复杂的两阶段处理流程，包括粗粒度和细粒度的自然语言处理（NLP）模块，如图 3 所示。在初始的粗粒度阶段，系统生成一组候选实体，包括时间、地点、对象、关系、移动对象标识符和最近邻数量。随后，在细粒度阶段，利用预先构建的知识库对候选实体集进行修剪，并提取精确实体。

4.1 自然语言候选实体

在这个阶段，我们的主要目标是构建自然语言候选集。效率是一个关键考虑因素，这需要快速识别实体信息。固有的

语义结构的复杂性需要使用先进的自然语言处理工具来近似和定位潜在的实体。

考虑到快速处理的必要性，我们选择 spaCy 作为主要的自然语言处理工具，NLTK 作为潜在的替代方案。我们框架的初始实现主要依赖于 spaCy 强大的分词和命名实体识别能力。使用 spaCy，我们构建了两个主要集合：(i) 信息集合 和 (ii) 数字集合。信息集合包含候选位置、对象和关系。数字集合包括最近邻数量、移动对象标识符和时间信息。

鉴于数字集合包含数值和时间信息，我们可以精确地定位时间和部分数字。例如，当遇到包含冒号的时间表达式，如“8:00”，我们使用正则表达式进行精确匹配和提取。对于缺少冒号的小时表达式，如“8am”，我们也使用正则表达式进行匹配和提取。最近邻的数量可以根据“最近”等关键词的语义距离来确定。由于查询对象可能存在模糊性，对象识别需要进一步验证。

4.2 数据库候选实体

将自然语言实体信息与确定性的数据库信息对齐存在重大挑战，因为自然语言本身存在固有的歧义和不确定性。为了解决这一问题，我们提出构建数据库候选项以实现精确的实体提取。为了实现位置、对象、关系和对象标识符的准确提取，我们提出了一个复杂的移动对象知识库（MOKB）。MOKB 由两个子知识库组成：(i) 关系知识库（RKB）和 (ii) 位置知识库（LKB）。

我们对数据库中的对象进行全面分析，以识别包含 mpoint 属性的关系，该属性用于表示运动对象。该关系封装了目标运动对象。LKB 主要从数据库中提取具有点、线和区域等属性的对象。鉴于同义词、缩写和位置歧义术语带来的复杂性，我们利用 GPT-4o 的能力来增强现有 LKB，添加潜在的同义词、缩写和歧义表达。扩展的术语仅用于与原始位置进行实体匹配，并不会作为最终位置实体呈现。RKB 用于确定运动对象的关系和对象。一旦识别出对象，将从数字集中确定与该对象语义距离最近且之前未被识别为其他实体的数值作为对象标识符。

在管理大规模数据库时，知识库可能显著扩展，导致检索过程中耗时过长。为缓解这一问题，我们为MOKB设计了一个分层索引以加快检索操作。

5 结构化语言生成

移动对象查询的固有复杂性使得开发通用结构化语言模型（SLM）不切实际

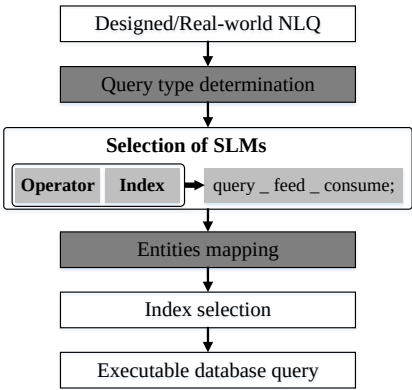


图5：结构化语言构建。

并且代价高，尤其是在现实场景中。因此，我们提出了一种分而治之的策略以降低模型设计成本，并开发了一种查询优化模型，通过索引选择来优化生成的结构化语言，其详细构建过程如图5所示。

5.1 结构化语言模型概要与查询类型分类

图 5 描述了草图 SLM，其中 $\langle k \rangle$ 、 $\langle t \rangle$ 、 $\langle o_i \rangle$ 、 $\langle place \rangle$ 和 $\langle relation \rangle$ 分别表示未确定数量的最近邻、时间、对象和标识符、位置以及移动对象关系。在一个位置包含在移动对象关系中的场景中，模型中的 $\langle place \rangle$ 占位符会被以下表达式替换。这里， $\langle tmp relation \rangle$ 和 $\langle name \rangle$ 分别表示移动对象关系和对象名称：“ $\langle tmp relation \rangle$ feed filter [$.Name = \langle name \rangle$] extract [Trip]”。表 1 列举了部分关键操作符，包括名称、签名和功能。

表1：SECONDO中的操作符。

Operator	Signature	Meaning
filter	$stream \times filter \ condition \rightarrow stream$	Filter the elements of the stream by a predicate.
trajectory	$mpoint \rightarrow line$	Map continuous moving points into trajectories.
deftime	$mpoint \rightarrow periods$	Project motion onto the time dimension.
at	$mpoint \times \{point, region\} \rightarrow mpoint$	Return moving points to a certain point or region.
knearest	$stream \times mpoint \times int \ k \rightarrow stream$	Compute the k nearest neighbors.
dist_euclidean	$pointseq \times pointseq \rightarrow real$	Compute the trajectory similarity.

对于类型分类模型的训练，语料库被按照8:2的比例划分为训练集和测试集。LSTM被用于训练，训练好的模型可以用来确定输入自然语言查询（NLQ）的查询类型。

5.2 结构化语言模型选择与槽位填充

对于不同的查询类型，我们使用预定义的映射规则将相应的实体映射到各自的槽位并将它们组合起来

使用所需的操作符来生成结构化语言。基于NALMO [13] 中描述的方法，可以构建用于时间区间查询、范围查询和最近邻查询的SLM。在本文中，我们重点关注相似性阈值、索引的使用以及实际场景中结构化语言的构建。由于篇幅有限，我们仅以跨区域查询为例。SLM的详细描述见NALSpatial和NALMO。

为了确定出租车轨迹是否与运营边界相交以及是否位于运营区域内，我们使用 `intersects` 操作符。如果存在相交或轨迹位于运营区域之外，我们可以推断存在跨区域运营。然而，如果出租车在短时间内离开运营边界后又重新进入，这种行为不会被视作跨区域运营。

6 实验结果与讨论

我们在本节中报告了大量实验结果。实现是在一台运行 Ubuntu 20.04 (64 位, 内核版本 5.14.0-1051-oem) 的计算机上开发的, 该计算机配备 Intel(R) Core(TM) i9-10850K CPU, 3.60 GHz, 32 GB 内存, 512 GB 硬盘。我们的目标是研究现有的文本到 SQL 方法和大型语言模型在解决文本到 EXE 问题中的表现, 重点关注它们在 MODs 领域中的适用性和鲁棒性。为了确保全面评估, 我们不仅评估翻译准确性, 还评估生成的可执行语言的质量, 包括翻译效率和执行效率。我们提供了对不同方法优缺点的深入理解, 为实际应用提供有价值的见解。

6.1 实验设置

设置。如第3节所述, 采用手动和基于大语言模型 (LLM) 的注释相结合的方法。虽然利用LLM进行生成可以减轻工作量, 但仍然需要一定程度的人工审查。因此, 最终对600对自然语言查询 (NLQs) 和结构化查询对进行了注释。示例对1可用。现有查询主要分为两类: (i) 专门为数据库设计的查询, (ii) 适用于现实世界需求的查询, 这两类均被考虑。专用查询包括时间间隔查询 (TI)、范围查询 (RA)、最近邻查询 (NN)、连接查询 (J) 和轨迹相似性查询 (TS), 而现实场景查询则包括跨区域查询 (CR) 和绕行查询 (DT)。根据查询长度, Spider将SQL查询难度分为四个级别, 包括简单、中等、困难和极难。在此基础上, 我们进一步将移动对象查询分为三类。

数据集。本实验使用的数据集包括真实世界的出租车、汽车和公共汽车移动轨迹, 以及系统模拟的火车移动轨迹, 如表2所示, 以确保实验结果的全面性和准确性。真实世界的轨迹数据集nanjingtest覆盖了2024年6月15日00:00至24:00的时间段, 记录了302个移动对象的移动片段, 总共包含917,006条数据。

表 2: 基准测试中使用的数据集详情。

Dataset	#Points	#Lines	#Regions	#Moving objects
nanjingtest	9000	887	13	302
berlintest	3040	4078	330	562

移动目标点。与此同时, 系统模拟的列车轨迹数据集berlintest覆盖了2020年11月20日06:00到09:00的时间段, 涉及来自562列火车的轨迹片段, 共包含51,544个移动目标点。

6.2 评估指标

在文本到 SQL 的领域中, 系统性能的评估主要依赖于三个关键指标: (i) 完全匹配 (EM), (ii) 执行准确率 (EX), 以及 (iii) 有效效率评分 (VES) [7]。我们的评估指标受到上述三个指标的启发。

为了评估文本到可执行文件系统的自然语言理解能力, 我们提出了可翻译性 (TA) 这一指标。给定系统生成的可执行数据库查询集合和输入自然语言问题集合, 分别记为 EQ 和 N, TA 定义如下:

$$TA = \frac{|EQ|}{|N|} \quad (1)$$

该指标用于评估可执行语言的语义完整性, 这些语言在逻辑上正确且实体完整。TA通过承认所有结构上可行的数据库查询来扩大范围, 前提是可执行查询包含所需实体并保持逻辑完整性。

此外, 为了评估在构建结构化语言方面的熟练程度, 我们提出了翻译精确度 (TP) 指标。在与预期结果和输入自然语言查询 (NLQs) N 对齐的可执行数据库查询集合上定义, TP 评估可执行语言在执行后是否能够产生预期输出。我们比较生成的可执行数据库查询与黄金查询的结果, 假设相同的执行结果表示两者等价。设 S_n 为第 n 个示例的可执行数据库查询集合, G_n 为第 n 个示例的黄金数据库查询集合, $R(S_n)$ 为执行 S_n 后的结果集, $R(G_n)$ 为执行 G_n 后的结果集, 则 TP 可以定义为:

$$TP = \frac{1}{N} \sum_{n=1}^N (\mathbb{1}(S_n, G_n) \cdot \mathbb{1}(R(S_n), R(G_n))) \quad (2)$$

其中 $\mathbb{1}(A, B)$ 是定义如下的指示函数:

$$\mathbb{1}(A, B) = \begin{cases} 1, & \text{if } A = B \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

为了全面评估生成的可执行语言的质量, 我们的分析扩展到包括称为翻译效率得分 (Translation Efficiency Score, TES) 的效率指标, 包括执行时间和在执行过程中 CPU 时间的综合测量。TES 超越了 VES, 通过考虑执行时间和 CPU 时间, 从而提供更全面的评估维度。我们只考虑能够产生与给定可执行语言相同结果的语句。令 $E(\cdot)$ 表示执行时间的测量, $C(\cdot)$ 表示

¹<https://pan.nuaa.edu.cn/share/c9bd35cbe21c175137d5de4db9>.

Table 3: Results of TA and TP.

Method	TA/%								TP/%							
	Easy	Medium				Hard		All	Easy	Medium				Hard		All
	TI	RA	NN	J	TS	CR	DT		TI	RA	NN	J	TS	CR	DT	
NaLIR	5.84	3.06	1.79	1.05	2.00	0.00	0.00	2.52	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
ATHENA++	37.27	31.61	21.43	27.37	28.00	10.17	23.53	29.87	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
ValueNet	35.04	20.41	19.64	18.00	16.95	11.86	15.69	24.33	16.79	14.29	10.71	5.26	2.00	3.39	5.88	9.23
LLaMA3-70B (few-shot)	51.10	33.16	30.17	22.11	16.50	10.17	31.37	30.20	23.36	19.47	20.54	10.00	5.50	5.93	6.86	14.18
GPT-4o (few-shot)	68.98	44.75	45.54	30.53	35.50	11.87	37.26	42.87	47.45	34.19	35.72	20.53	9.50	7.63	18.65	27.01
NALMO+	89.05	79.78	78.57	73.68	77.00	45.76	58.82	68.96	72.99	61.22	57.14	61.05	65.00	37.29	41.18	60.07

TI, 时间间隔查询; RA, 范围查询; NN, 最近邻查询; J, 连接查询; TS, 轨迹相似性查询; CR, 跨区域; DT, 绕行。

CPU 时间的测量。这两个函数用于测量给定环境中的绝对时间。我们可以设计一个结合这两个函数的公式：

$$TES = \frac{\sum_{n=1}^N \left(\mathbb{1}(R(S_n), R(G_n)) \cdot \text{Cost}(Y_n, \hat{Y}_n) \right)}{N} \quad (4)$$

$$\text{Cost}(Y_n, \hat{Y}_n) = \frac{\sqrt{E(Y_n)}}{\sqrt{E(\hat{Y}_n)}} \cdot \frac{\sqrt{C(Y_n)}}{\sqrt{C(\hat{Y}_n)}} \quad (5)$$

其中 $\text{Cost}(\cdot)$ 表示生成的可执行语言相对于标准可执行语言的相对效率。我们全面考虑标准可执行语言与生成的可执行语言的执行时间和 CPU 时间比，以确保全面评估。

6.3 基线方法

我们对六个系统进行了比较分析。考虑到文本到SQL的快速发展，我们从三类流行方法中选择了具有代表性的方法：(i) 基于规则的方法。我们采用了NaLIR和ATHENA++。NaLIR是一种经典的基于规则的方法，利用解析树进行实体提取和对齐，并通过用户反馈构建SQL。ATHENA++通过两阶段生成方法，将OQL查询作为中间表示。(ii) 基于神经网络的方法。我们选择ValueNet [1]，它在IRNet的基础上扩展了SemQL语法，使生成的查询中包含值。(iii) 基于大语言模型的方法。我们利用GPT-4o和开源模型LLaMA3作为对比方法。在BIRD排行榜中，前十名的方法中有一半上都基于GPT-4o。考虑到LLaMA3是最广泛使用的开源模型之一。我们设计实验时使用了DAIL-SQL [2]的方法论，将模式信息集成到提示设计中。

6.4 实验结果

我们考虑查询翻译能力。表3展示了所有方法的TA和TP，提供了不同难度级别和查询类型的详细数据。由于MOD中的可执行语言在语法结构和语义表示上都有所不同

在标准 SQL 中，基于规则的方法仅专注于自然语言理解过程。

NaLIR 构建语义解析树，然后将节点映射到 SQL 片段。这种数据库预设方法在实体识别和操作符选择上产生高错误率。因此，NaLIR 获得较低的 TA，并且 TP 无法直接测量。ATHENA++ 结合了领域本体和 Stanford CoreNLP，用于简单查询的高级 NLP。简单查询涉及的实体较少。然而，ATHENA++ 无法提取或映射模糊实体，例如同义词、缩写或含糊术语。在实际场景查询中，自然语言通常提供的实体信息有限，并且需要多步骤分解，ATHENA++ 会完全失效。

ValueNet 发出中间表示 (IR)，这使得将 IR 转换为结构化 SQL 的后处理步骤显得至关重要。通过摄取实际数据库内容，ValueNet 达到了基于 CoreNLP 方法的 TA 水平。然而，由于 ValueNet 依赖大规模监督训练，其生成可执行语言的能力仍然有限。TP 在所有类别中均低于 20%，并且随着查询复杂性的增加而进一步下降。

为了分析大型语言模型 (LLMs) 的性能，我们设计了少量示例提示实验。由于运动对象可执行语言与标准 SQL 之间存在句法和语义差距，没有示例提示的 LLM 在所有查询任务中都失败。GPT-4o 在 TA 和 TP 上均优于 LLaMA3，分别显示出超过 50% 和 100% 的相对提升。然而，由于丰富的语义模式和缺乏固定的操作符实体模板，单次提示不足以应对未见过的查询，尽管生成的逻辑可能是正确的，但省略初始时间过滤步骤会导致执行速度比黄金可执行数据库查询慢。将示例数量增加到五个，为 LLaMA3 带来 TA 和 TP 超过 20% 的增益，为 GPT-4o 带来超过 40% 的增益。然而，进一步增加样本量可能会存在幻觉风险，包括操作符误用和虚构不存在的操作符。

对于 TES，ValueNet 的小模型和大语言模型 (LLMs) 都表现出次优性能。ValueNet、LLaMA3 和 GPT-4o 的得分分别为 12.43、26.37 和 60.71。由于可执行语言具有固有的白盒特性，查询效率可以被直接控制。小模型与训练数据集紧密绑定，而大语言模型没有接触过足够直接与数据库索引相关的训练数据，尤其是在使用索引优化查询方面。这需要大量的重新训练或微调。

大数据集。相比之下，混合方法 NALMO+ 展示了显著的优势，即使在大量数据集的情况下，也能保持相对高效的查询，达到了 68.37 的 TES。

总之，这些研究结果强调了NALMOBench基准所带来的挑战，进一步证明了文本到可执行文件的研究仍处于早期阶段，在该领域还有很大的探索和发展潜力。

6.5 实验讨论

查询复杂性主要由查询的内在难度决定。对于简单的 TI 查询，仅包含时间约束、目标对象和关系信息。结构化语言相对固定，仅使用时间操作符。因此，自然语言理解阶段和结构化语言生成阶段都保持简单，使得所有评估的方法都能在 TA 和 TP 上取得最高分。相比之下，中等复杂度查询，包括 RA、NN、J 和 TS 查询，需要更丰富的实体属性（例如时间、地点、关系、对象标识符和最近邻数量）。这种丰富的语义范围显著增加了自然语言解析的难度。同时，结构化语言的语法必须覆盖扩展的操作符词汇，导致操作符和实体配对出现组合爆炸。大语言模型表现出三大主要不足。

以 TS 查询为具体例子。相似度阈值必须在每个数据集上通过经验进行校准，这需要依赖特定数据集的实验和手动调整。然而，LLM 无法推断出最佳阈值，并且经常生成虚假的操作符或默认使用很少使用但有效的操作符，这会降低查询性能。在困难级别上，CR 和 DT 查询通常涉及多个连续处理步骤或嵌套查询结构。由于实际执行需要迭代查询处理，现有方法往往无法有效应对，TA 和 TP 指标都保持在 50% 以下。

7 结论

在本文中，我们介绍了 NALMOBench，这是一种新颖的基准，用于评估将自然语言查询（NLQs）转换为 MODs 可执行语言的能力和质​​量。NALMOBench 不仅覆盖了合成运动物体数据集，还涵盖了真实世界的轨迹数据。我们进一步提出了一种基于大语言模型（LLM）的生成领域特定数据库语料库的方法。该方法包括常规的运动物体查询以及来自复杂真实世界场景的查询。为了解决从文本到可执行语言（text-to-EXE）的问题，我们将问题拆分为两个子问题：（i）自然语言理解和（ii）结构化语言生成，并为每个阶段提供了基线算法。

我们的实验表明，NALMOBench 对现有的文本到可执行程序（text-to-EXE）方法和大型语言模型（LLM）构成了显著挑战。我们认为，NALMOBench 为在 MOD 领域评估文本到可执行系统建立了一个严格的新标准。NALMOBench 为应对实际轨迹数据使用的固有复杂性以及推动优化后翻译可执行语言效率的研究奠定了基础。

8 感谢

本工作得到江苏省研究生科研与实践创新计划（KYCX24_0608）以及南京航空航天大学博士生跨学科创新基金（KXKCXJJ202505）的资助。

参考文献

[1] Ursin Brunner 和 Kurt Stockinger. 2021. ValueNet: 一个从数据库信息学习的自然语言到 SQL 系统. 载于 ICDE. 2177–2182. [2] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, 和 Jingren Zhou. 2024. 基于大型语言模型的 Text-to-SQL: 基准评估. Proc. VLDB Endow. 17, 5 (2024), 1132–1145. [3] Jiaqi Guo, Zecheng Zhan, Yan Gao, 等. 2019. 面向跨领域数据库的复杂 Text-to-SQL 中间表示方法. 载于 ACL. 4524–4535. [4] Ralf Hartmut Güting, Thomas Behr, 和 Christian Düntgen. 2010. SECONDO: 移动对象数据库研究的平台及研究实现的发布与整合. IEEE Data Eng. Bull. 33, 2 (2010), 56–63. [5] Chia-Hsuan Lee, Oleksandr Polozov, 和 Matthew Richardson. 2021. KaggleDBQA: Text-to-SQL 解析器的现实评估. 载于 ACL/IJCNLP. 2261–2273. [6] Fei Li 和 H. V. Jagadish. 2014. 构建一个交互式自然语言界面